



INSTITUT TEKNOLOGI DEL

MATERI PRAKTIKUM

4233201 – Kecerdasan Buatan

Semester Genap Tahun Ajaran 2026/2027

Session Date	: 15 September 2026
Week/Session	: 2/3
Topic	: Data Preprocessing
Practicum Goals	: Mahasiswa mampu melakukan pra pemrosesan data seperti handling missing value dan outliers
Duration	: 2 x 50 minutes
Place of Submission	: ecourse
Submission Deadline	: Akhir Sesi 3

Missing Value

```
In [1]: import pandas as pd
path = "dataMissingValue.csv"
df = pd.read_csv(path)
df
```

```
Out[1]:
```

	Name	School Level	Blood Type	Age	Fav Fruit	Shoe Chart
0	A	High School	AB	15.0	apple	36.0
1	B	High School	NaN	17.0	banana	41.0
2	C	High School	NaN	NaN	watermelon	NaN
3	D	High School	NaN	16.0	NaN	38.0
4	E	Middle School	B	15.0	banana	37.0
5	F	Middle School	NaN	14.0	grapes	NaN
6	G	Middle School	O	NaN	mango	36.0
7	H	High School	A	16.0	papaya	39.0
8	I	High School	NaN	18.0	NaN	40.0
9	J	Middle School	NaN	NaN	avocado	35.0

Menghitung jumlah data yang hilang (NaN) di setiap kolom untuk memahami kualitas dataset.

```
In [2]: df.isnull().sum()
```

```
Out[2]: Name          0
School Level  0
Blood Type    6
Age           3
Fav Fruit     2
Shoe Chart    2
dtype: int64
```

Menghitung persentase data yang hilang untuk mengevaluasi apakah perlu dihapus atau diisi ulang.

```
In [3]: percent_missing = 100*df.isnull().mean()
percent_missing
```

```
Out[3]: Name          0.0
        School Level  0.0
        Blood Type    60.0
        Age           30.0
        Fav Fruit      20.0
        Shoe Chart     20.0
        dtype: float64
```

df.dropna digunakan untuk menghapus baris atau kolom yang memiliki missing values.

```
In [4]: df.dropna(axis=0)
```

```
Out[4]:
```

	Name	School Level	Blood Type	Age	Fav Fruit	Shoe Chart
0	A	High School	AB	15.0	apple	36.0
4	E	Middle School	B	15.0	banana	37.0
7	H	High School	A	16.0	papaya	39.0

```
In [5]: df.dropna(axis=1)
```

```
Out[5]:
```

	Name	School Level
0	A	High School
1	B	High School
2	C	High School
3	D	High School
4	E	Middle School
5	F	Middle School
6	G	Middle School
7	H	High School
8	I	High School
9	J	Middle School

thresh = 5 berarti hanya baris yang memiliki minimal 5 nilai valid yang dipertahankan. Contohnya baris indeks 2 hilang karena pada indeks tersebut nilai yang tidak berisi NaN kurang dari 5.

```
In [6]: df.dropna(thresh=5)
```

Out[6]:

	Name	School Level	Blood Type	Age	Fav Fruit	Shoe Chart
0	A	High School	AB	15.0	apple	36.0
1	B	High School	NaN	17.0	banana	41.0
4	E	Middle School	B	15.0	banana	37.0
6	G	Middle School	O	NaN	mango	36.0
7	H	High School	A	16.0	papaya	39.0

Dengan menambahkan subset 'Age', maka hanya baris yang berisi NaN di kolom Age yang dihapus.

In [7]: `df.dropna(subset=['Age'])`

Out[7]:

	Name	School Level	Blood Type	Age	Fav Fruit	Shoe Chart
0	A	High School	AB	15.0	apple	36.0
1	B	High School	NaN	17.0	banana	41.0
3	D	High School	NaN	16.0	NaN	38.0
4	E	Middle School	B	15.0	banana	37.0
5	F	Middle School	NaN	14.0	grapes	NaN
7	H	High School	A	16.0	papaya	39.0
8	I	High School	NaN	18.0	NaN	40.0

Menghapus kolom Blood Type dengan df.drop()

In [8]: `df_drop = df.drop(columns = 'Blood Type')`
`df_drop`

Out[8]:

	Name	School Level	Age	Fav Fruit	Shoe Chart
0	A	High School	15.0	apple	36.0
1	B	High School	17.0	banana	41.0
2	C	High School	NaN	watermelon	NaN
3	D	High School	16.0	NaN	38.0
4	E	Middle School	15.0	banana	37.0
5	F	Middle School	14.0	grapes	NaN
6	G	Middle School	NaN	mango	36.0
7	H	High School	16.0	papaya	39.0
8	I	High School	18.0	NaN	40.0
9	J	Middle School	NaN	avocado	35.0

.fillna() digunakan untuk mengisi nilai yang kosong (missing values). Imputasi nilai bisa menggunakan beberapa metode statistik seperti mean, median, mode.

Missing value pada kolom Age akan diisi dengan nilai rata-rata atau mean, Shoe Chart dengan median, dan Fav Fruit dengan mode.

```
In [9]: df_filled = df_drop.copy()
df_filled['Age'] = df['Age'].fillna(df['Age'].mean())
df_filled
```

Out[9]:

	Name	School Level	Age	Fav Fruit	Shoe Chart
0	A	High School	15.000000	apple	36.0
1	B	High School	17.000000	banana	41.0
2	C	High School	15.857143	watermelon	NaN
3	D	High School	16.000000	NaN	38.0
4	E	Middle School	15.000000	banana	37.0
5	F	Middle School	14.000000	grapes	NaN
6	G	Middle School	15.857143	mango	36.0
7	H	High School	16.000000	papaya	39.0
8	I	High School	18.000000	NaN	40.0
9	J	Middle School	15.857143	avocado	35.0

```
In [10]: df_filled['Shoe Chart'] = df['Shoe Chart'].fillna(df['Shoe Chart'].median())
df_filled
```

Out[10]:

	Name	School Level	Age	Fav Fruit	Shoe Chart
0	A	High School	15.000000	apple	36.0
1	B	High School	17.000000	banana	41.0
2	C	High School	15.857143	watermelon	37.5
3	D	High School	16.000000	NaN	38.0
4	E	Middle School	15.000000	banana	37.0
5	F	Middle School	14.000000	grapes	37.5
6	G	Middle School	15.857143	mango	36.0
7	H	High School	16.000000	papaya	39.0
8	I	High School	18.000000	NaN	40.0
9	J	Middle School	15.857143	avocado	35.0

```
In [11]: df_filled['Fav Fruit'] = df['Fav Fruit'].fillna(df['Fav Fruit'].mode().iloc[0])
df_filled
```

Out[11]:

	Name	School Level	Age	Fav Fruit	Shoe Chart
0	A	High School	15.000000	apple	36.0
1	B	High School	17.000000	banana	41.0
2	C	High School	15.857143	watermelon	37.5
3	D	High School	16.000000	banana	38.0
4	E	Middle School	15.000000	banana	37.0
5	F	Middle School	14.000000	grapes	37.5
6	G	Middle School	15.857143	mango	36.0
7	H	High School	16.000000	papaya	39.0
8	I	High School	18.000000	banana	40.0
9	J	Middle School	15.857143	avocado	35.0

Selain itu, missing value bisa juga diisi dengan back fill dan forward fill. Back fill untuk mengisi nilai kosong di "Shoe Chart" dengan nilai dari bawahnya atau setelahnya. Forward fill ntuk mengisi nilai kosong di "Fav Fruit" dengan nilai sebelumnya.

```
In [12]: df_filled2 = df_drop.copy()
df_filled2['Shoe Chart'] = df_filled2['Shoe Chart'].replace("", None).bfill()
df_filled2
```

Out[12]:

	Name	School Level	Age	Fav Fruit	Shoe Chart
0	A	High School	15.0	apple	36.0
1	B	High School	17.0	banana	41.0
2	C	High School	NaN	watermelon	38.0
3	D	High School	16.0	NaN	38.0
4	E	Middle School	15.0	banana	37.0
5	F	Middle School	14.0	grapes	36.0
6	G	Middle School	NaN	mango	36.0
7	H	High School	16.0	papaya	39.0
8	I	High School	18.0	NaN	40.0
9	J	Middle School	NaN	avocado	35.0

```
In [13]: df_filled2['Fav Fruit'] = df_filled2['Fav Fruit'].replace("", None).ffill()
df_filled2
```

Out[13]:

	Name	School Level	Age	Fav Fruit	Shoe Chart
0	A	High School	15.0	apple	36.0
1	B	High School	17.0	banana	41.0
2	C	High School	NaN	watermelon	38.0
3	D	High School	16.0	watermelon	38.0
4	E	Middle School	15.0	banana	37.0
5	F	Middle School	14.0	grapes	36.0
6	G	Middle School	NaN	mango	36.0
7	H	High School	16.0	papaya	39.0
8	I	High School	18.0	papaya	40.0
9	J	Middle School	NaN	avocado	35.0

Outliers

```
In [14]: path = "weight-height_outlier.csv"
df2 = pd.read_csv(path)
df2
```

Out[14]:

	Gender	Height	Weight
0	Male	73.847017	241.893563
1	Male	68.781904	162.310473
2	Male	74.110105	212.740856
3	Male	71.730978	220.042470
4	Male	69.881796	206.349801
...	...	...	...
9995	Female	66.172652	136.777454
9996	Female	67.067155	170.867906
9997	Female	63.867992	128.475319
9998	Female	69.034243	163.852461
9999	Female	61.944246	113.649103

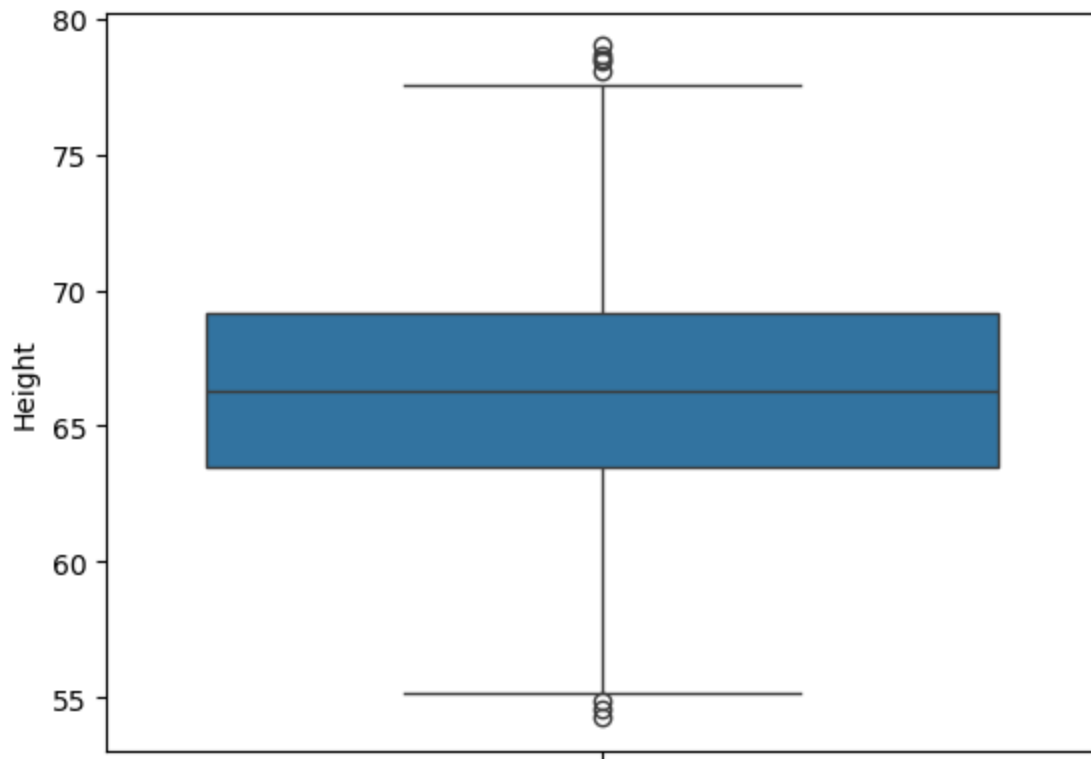
10000 rows × 3 columns

Mendeteksi outliers secara visual dengan menggunakan boxplot dari seaborn dan menghapus outliers dengan menggunakan Interquartile Range.

Outliers berada di luar titik whiskers pada boxplot.

```
In [15]: import seaborn as sns
sns.boxplot(df2['Height'])
```

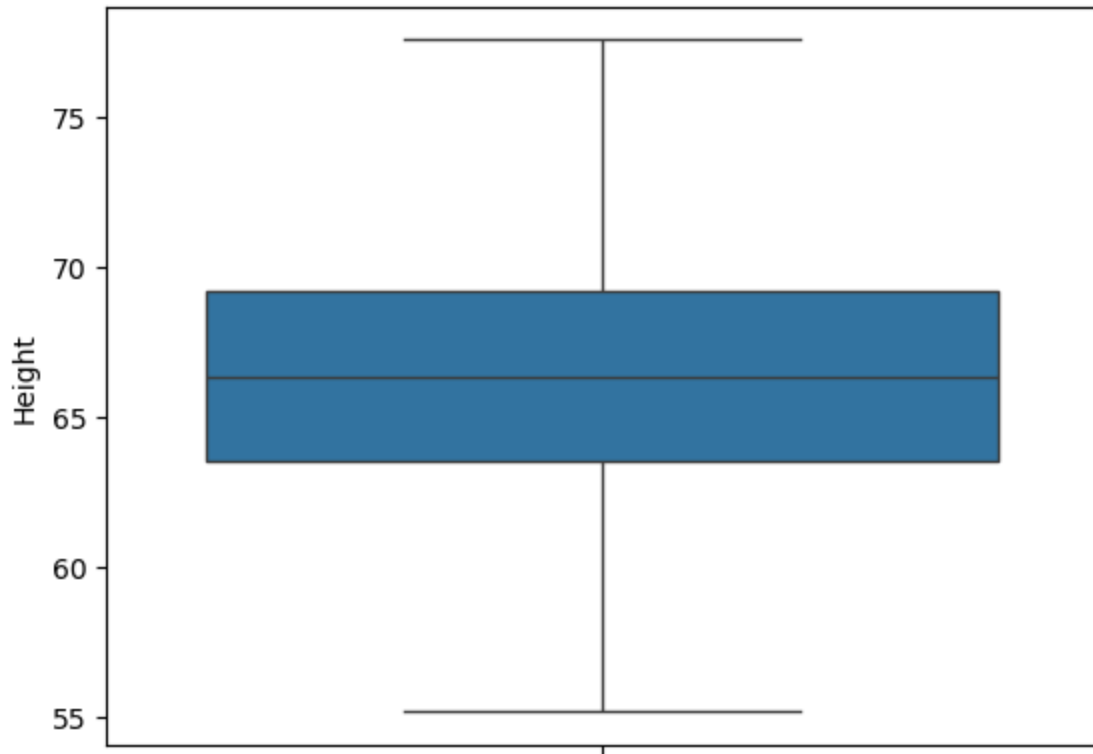
Out[15]: <Axes: ylabel='Height'>



```
In [16]: import numpy as np
# Identifikasi Q1 Q3
q1, q3 = np.percentile(df2['Height'], [25, 75])
# Hitung IQR
iqr = q3 - q1
# Cari lower dan upper bounds
lower_bound = q1 - (1.5 * iqr)
upper_bound = q3 + (1.5 * iqr)
# Hapus outliers
clean_data = df2[(df2['Height'] >= lower_bound)
                  & (df2['Height'] <= upper_bound)]

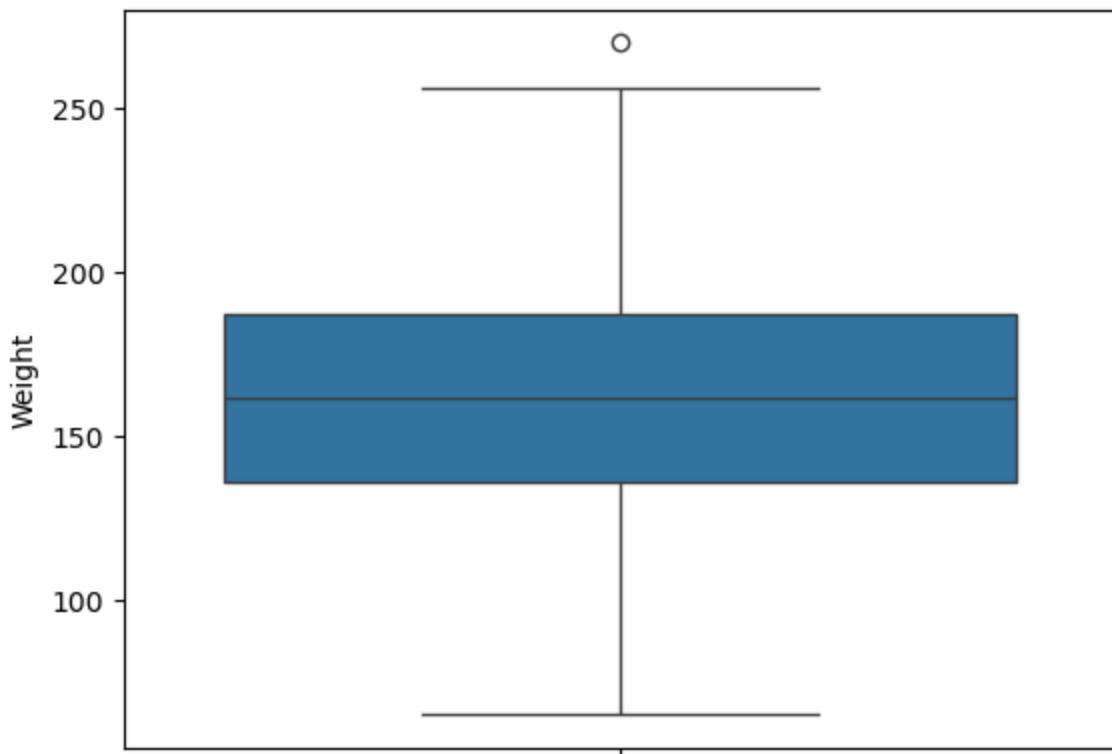
sns.boxplot(clean_data['Height'])
```

```
Out[16]: <Axes: ylabel='Height'>
```



```
In [17]: sns.boxplot(df2['Weight'])
```

```
Out[17]: <Axes: ylabel='Weight'>
```

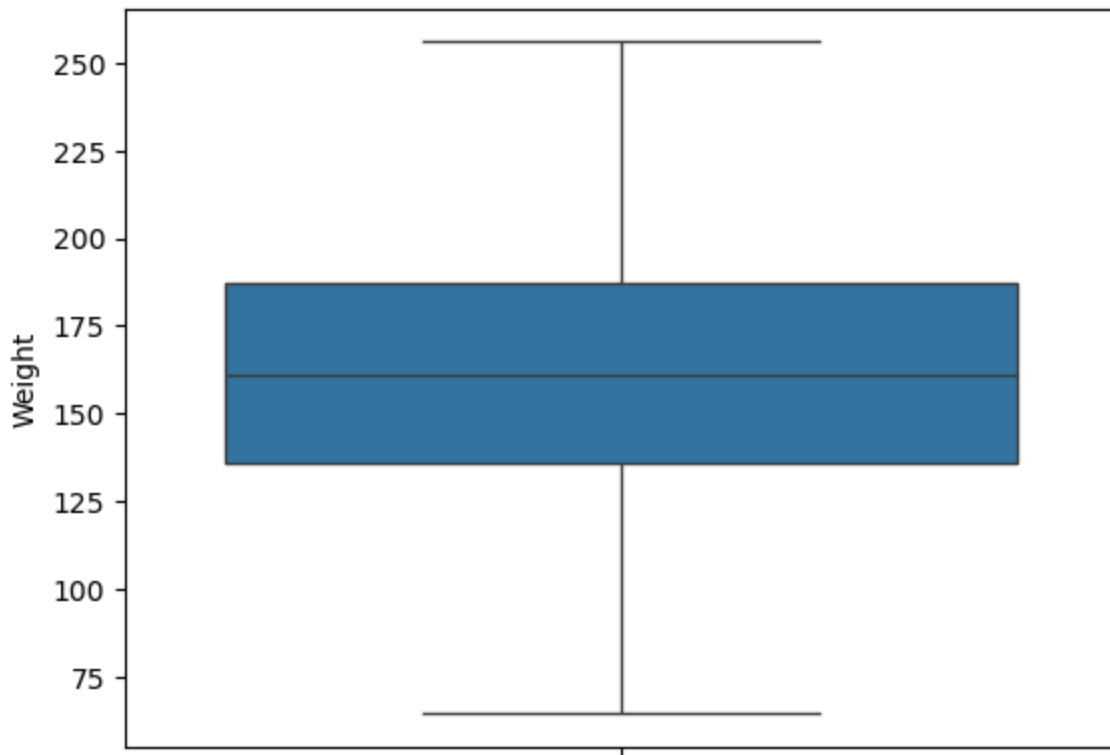


```
In [18]: # Identifikasi Q1 Q3
q1, q3 = np.percentile(df2['Weight'], [25, 75])
# Hitung IQR
iqr = q3 - q1
```

```
# Cari lower dan upper bounds
lower_bound = q1 - (1.5 * iqr)
upper_bound = q3 + (1.5 * iqr)
# Hapus outliers
clean_data = df2[(df2['Weight'] >= lower_bound)
                  & (df2['Weight'] <= upper_bound)]

sns.boxplot(clean_data['Weight'])
```

Out[18]: <Axes: ylabel='Weight'>



Mendeteksi outliers dengan visualisasi dari matplotlib.

```
In [19]: import matplotlib.pyplot as plt

# Membuat daftar kolom yang ingin dianalisis, yaitu "Weight" dan "Height".
columns_to_plot = ['Weight', 'Height']

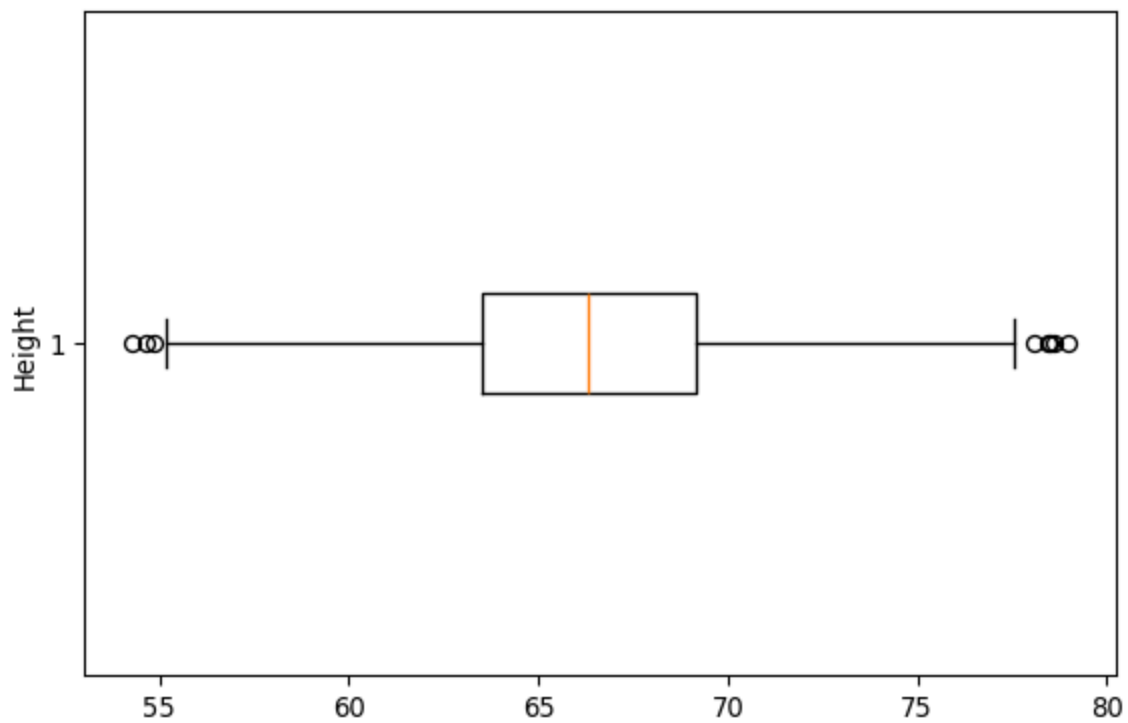
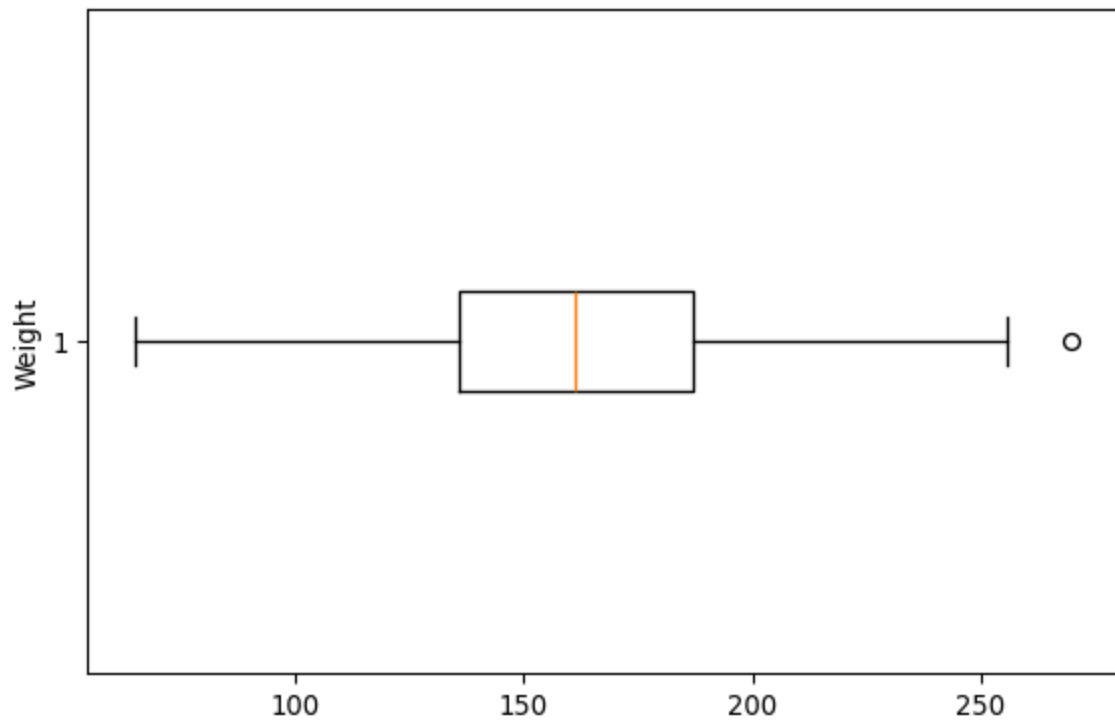
# Memastikan kolom ada di dataset
columns_to_plot = [col for col in columns_to_plot if col in df2.columns]

fig, axs = plt.subplots(len(columns_to_plot), 1, dpi=95, figsize=(7, 5 * len(column

if len(columns_to_plot) == 1:
    axs = [axs]

for i, col in enumerate(columns_to_plot):
    axs[i].boxplot(df2[col], vert=False)
    axs[i].set_ylabel(col)

plt.show()
```



```
In [20]: def remove_outliers_sequentially(df2, columns):
df_cleaned = df2.copy()
for col in columns:
    q1, q3 = np.percentile(df_cleaned[col], [25, 75])
    iqr = q3 - q1
    lower_bound = q1 - (1.5 * iqr)
    upper_bound = q3 + (1.5 * iqr)
    df_cleaned = df_cleaned[(df_cleaned[col] >= lower_bound) & (df_cleaned[col]
return df_cleaned
```

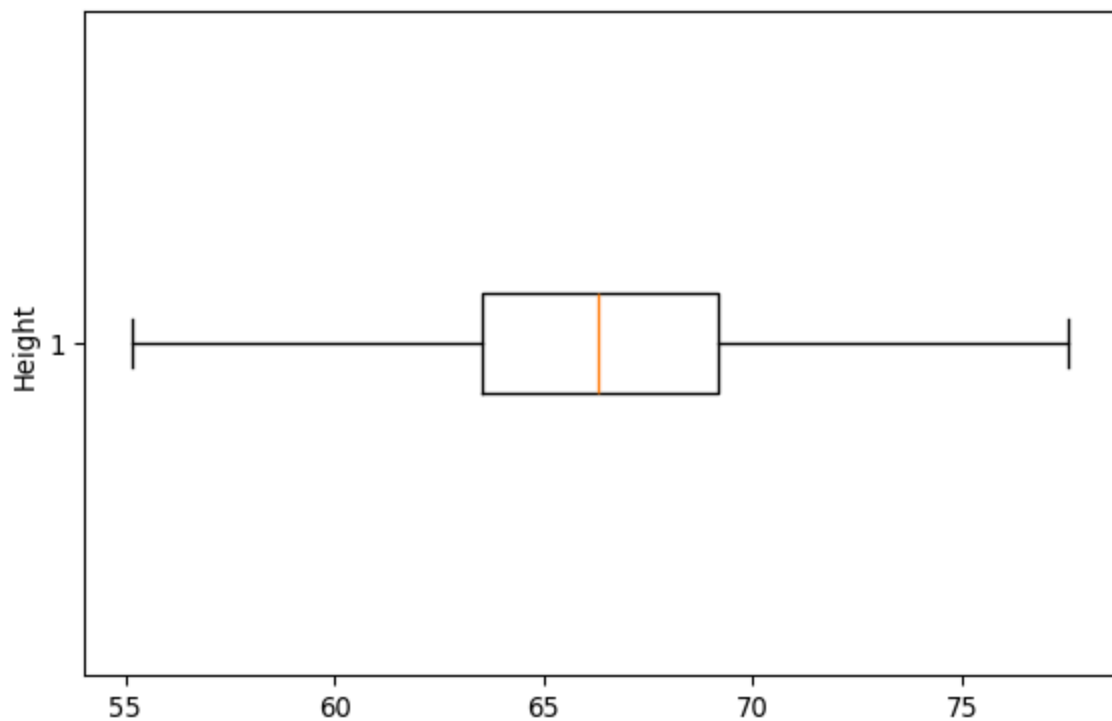
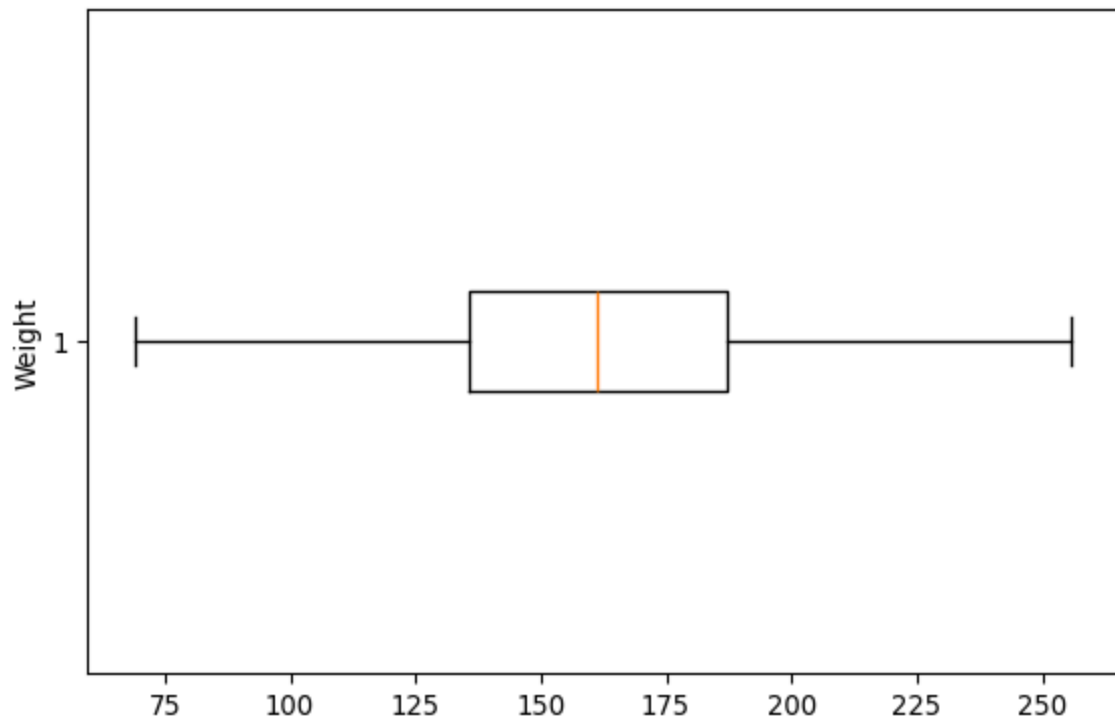
```
# Terapkan fungsi untuk menghapus outliers dari "Height" dan "Weight"
clean_data = remove_outliers_sequentially(df2, ['Height', 'Weight'])

# Plot boxplots setelah menghapus outliers
columns_to_plot = ['Weight', 'Height']
fig, axs = plt.subplots(len(columns_to_plot), 1, dpi=95, figsize=(7, 5 * len(columns_to_plot)))

if len(columns_to_plot) == 1:
    axs = [axs]

for i, col in enumerate(columns_to_plot):
    axs[i].boxplot(clean_data[col], vert=False)
    axs[i].set_ylabel(col)

plt.show()
```



```
In [21]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))

# Memisahkan kolom numerik dan non-numerik
numeric_cols = clean_data.select_dtypes(include=[np.number]).columns # Ambil kolom
non_numeric_cols = clean_data.select_dtypes(exclude=[np.number]).columns # Ambil k

rescaled_numeric = scaler.fit_transform(clean_data[numeric_cols])
rescaled_numeric[:]
```

```
Out[21]: array([[0.83480376, 0.92524773],
                [0.6086688 , 0.49939825],
                [0.84654949, 0.76925146],
                ...,
                [0.38928431, 0.31834619],
                [0.61993463, 0.50764944],
                [0.30339752, 0.23901104]])
```

```
In [22]: # Konversi kembali hasil scaling ke DataFrame
rescaled_df = pd.DataFrame(rescaled_numeric, columns=numeric_cols, index=clean_data)

# Gabungkan kembali dengan kolom non-numerik tanpa mengubah jumlah baris
final_df = pd.concat([clean_data[non_numeric_cols], rescaled_df], axis=1)

final_df
```

```
Out[22]:
```

	Gender	Height	Weight
0	Male	0.834804	0.925248
1	Male	0.608669	0.499398
2	Male	0.846549	0.769251
3	Male	0.740332	0.808322
4	Male	0.657774	0.735053
...	...	...	...
9995	Female	0.492177	0.362771
9996	Female	0.532113	0.545189
9997	Female	0.389284	0.318346
9998	Female	0.619935	0.507649
9999	Female	0.303398	0.239011

9992 rows × 4 columns

Tugas

- Ikuti semua step dalam modul ini
- Handling missing value dari dataset 'Titanic-Dataset.csv'
 - Tampilkan `df.isnull().sum()` ketika semua missing value sudah diatasi
- Handling outliers dari dataset 'diabetes.csv'
 - Pilih 3 kolom untuk deteksi
 - Bebas menggunakan matplotlib atau seaborn